

Predictive Evaluation Competition

April 20, 2026

1 Introduction

Modern AI systems are evaluated using public benchmarks—MMLU, GSM8K, HumanEval, TerminalBench, and many others—designed to assess reasoning, knowledge, and domain-specific skills. A benchmark is often a convenient measurement instrument, but it is usually not the end goal. We read model performance results and infer their performance in other settings—on contemporaneous or future benchmarks, on downstream applications, or on tasks that have not yet been designed. This evaluation problem is a *predictive* problem. Despite its relevance, modern AI evaluation rarely uses predictive evaluation models. The AI community lacks a shared predictive benchmark for evaluating evaluation methods. This competition addresses that gap.

Throughout this handbook we use the psychometric terms *subject* and *item*, which may be unfamiliar to students coming from a pure machine-learning background. A **subject** is an AI system under evaluation (e.g., GPT-4, Claude, or Llama-3); the subject plays the role of the “test-taker.” An **item** is a single benchmark question or task the subject is asked to answer; items play the role of “test questions.” A **response** is a binary signal in $\{0, 1\}$, encoded as 1 if the subject answered the item correctly (pass) and 0 otherwise (fail). Each response also carries two contextual fields: the **benchmark** the item came from (e.g., MMLU) and the **test condition** under which it was administered (e.g., zero-shot, chain-of-thought). The goal of a predictive evaluator is to predict the probability of a correct response on a (subject, item) pair under a given (benchmark, condition), *without* actually running the subject on the item — and those four fields (subject, item, benchmark, condition) are exactly the inputs your submission will receive at test time.

Cold-start prediction on new items is the central problem in this competition. Each evaluation round places the submission in the hardest regime for a classical factor model: the test pairs involve items that have no observed responses in the training matrix — often items from benchmarks that did not yet exist when the training data was collected. The subjects in the test set are the same subjects present in the training matrix; the cold-start dimension is the item side alone. This mirrors the deployment pattern that matters most in practice: once a frontier model is released it tends to be evaluated on many public benchmarks, so the subject side of the matrix fills in relatively quickly, but proprietary datasets, newly authored benchmark questions, and entirely new benchmarks appear continuously, and each one is an unlabeled column of the response matrix. A law firm that wants to know how its LLM performs on its internal case-file questions, or a benchmark designer deciding which new questions will best separate frontier models, is asking an item cold-start question. Standard matrix completion has nothing to fit on these columns, so the only way to make predictions is to use side information: the item’s text, the subject’s metadata (size, family, release date, training recipe), and the benchmark and condition under which the evaluation is run. Given these four fields, can we predict the probability that the subject answers the item correctly, without ever running the subject on the item?

The competition also models a realistic second ingredient of such deployments: a small budget of ground-truth labels. A deployed predictive evaluator rarely operates in the zero-label limit — the same law firm that asks the item cold-start question is usually willing to hand-label a handful of its internal questions first, to pin the evaluator’s predictions to reality before trusting the rest. Each evaluation round therefore includes an optional *adaptive-labeling* channel: your submission names a few items whose ground-truth labels it most wants revealed, the platform discloses those labels, and your predictions on the remaining test items are allowed to use them. This is the active-learning loop that Lecture 5 (Sample-Efficient Measurement) is devoted to, embedded directly in the competition protocol. A submission can ignore this channel entirely and still produce valid predictions, but shipping a well-designed acquisition function is one of the main levers for winning on the leaderboard.

This challenge operationalizes the Lecture 4 discussion of Cold-Start and Prediction-Guided Evaluation into an end-to-end project. You can think of a strong submission as an instance of the three-stage PGE pipeline introduced in that lecture: (Stage 1) extract latent ability and item parameters from the observed training responses; (Stage 2a) learn a map from item text to item parameters; (Stage 2b) learn a map from subject metadata to ability; (Stage 3) combine the two at test time to produce a probability for any subject-item pair. Because every test item is new but every test subject has appeared in the training matrix, Stage 2a — the item-side content-to-parameter map — is the only lever that must generalize at test time; Stage 2b is optional, and a simple lookup of the Stage 1 ability estimates $\hat{U}_i$ for known subjects is already a reasonable baseline. The competition fixes the inputs (the four text fields) and the metric (negative log-loss on held-out pairs, higher-is-better), but leaves the modeling choices wide open — you can use factor models, neural IRT, transformer encoders, graph networks, or a content-based neural collaborative filter, and you can extend the framework with adaptive labeling (the subject of Lecture 5). You will practice the full workflow of a research-grade evaluation pipeline — data ingestion, model training and validation, held-out leaderboard iteration, and technical writing — and each submitted team leaves with a ranked leaderboard entry and a short NeurIPS-style paper you can cite in applications for research roles, graduate programs, or ML engineering positions.

2 Competition Structure

2.1 Data Format, Test Set, and Metrics

We provide real-world AI evaluation datasets that can optionally be used to train the predictive model. These datasets are on Huggingface: <https://huggingface.co/datasets/aims-foundations/measurement-db>. The data is in long format, with the following fields: item description, subject description, benchmark name, test condition (if any), and response. The response is always a binary scalar, encoded as 1 if the subject answered the item correctly (pass) and 0 otherwise (fail). The participant is encouraged to curate additional training data as they see fit.

The test set has the same format as the training set. The test set is strictly held out and never released to participants. During test time, each participant’s program receives the test item text, benchmark name, condition, and subject description for one hidden model-item pair at a time; only the ground-truth label is hidden. Data categories are used by the platform for sampling and adaptive-label budgets, not passed as an input field. The predictive program may use item content (e.g., embeddings, parsed question text) as predictive features. Each evaluation round scores predictions on a random subset of the held-out entries, rather than the full test set. Because the subset changes every round, repeated submissions cannot be used to reverse-engineer the test labels.

Internally, the hidden snapshot treats the same upstream benchmark item under different normalized test conditions as different item variants. Null, missing, and blank conditions are

normalized to an empty internal condition value; participant inputs render that value as "none". At runtime, submissions still receive ordinary (model_id, item_id) pairs: the item_id assigned in test_pairs.csv is already condition-specific, and the corresponding condition is exposed separately as input["condition"]. The prediction file format is unchanged: track1_predictions.csv has exactly model_id, item_id, and predicted_probability. Scoring still matches predictions to labels by (model_id, item_id) because that runtime item identifier already names the condition-specific variant.

Each submission is scored on predictions for the model-item pairs induced by $N = 5,000$ item variants sampled from the hidden pool. Items are organized into data categories, where each data category groups related benchmarks. The active configuration uses every available data category and samples items in a stratified way so that each data category contributes an approximately equal share, subject to the number of available hidden item variants in that category. This ensures broad coverage of evaluation domains and remains well-defined as the number of categories grows over time. Submissions are ranked by *negative log-loss* as the primary metric: the mean log-likelihood of the ground-truth labels under the submission's predicted probabilities, where *higher is better*. AUC-ROC is reported as a secondary metric (also higher-is-better) for debugging purposes. Both metrics are computed over the sampled hidden model-item pairs for each submission.

Teams are limited to 50 scored submissions per team per calendar day (UTC). The round-to-round resampling above is what prevents the hidden labels from being recovered by repeated leaderboard probes; the per-day cap exists to keep per-team compute cost bounded.

2.2 Adaptivity

In practice, a deployed predictive evaluator often has access to a small number of ground-truth labels. For example, a law firm evaluating an LLM on its proprietary data might run the model on a handful of curated questions and share the results. Given these labels, the evaluator can adapt its predictions to the target domain. To reward this capability, the competition provides an adaptive labeling mechanism. Each participant may submit an optional labeling.py that defines an acquisition function in the sense of active learning:

```
acquisition_function(input: dict) -> float
```

where input carries four string fields describing the candidate (benchmark, condition, subject content, item content), and the returned scalar indicates how desirable it is for the ground-truth label of this input to be revealed (higher means more desirable). The absolute scalar value is not considered, only the ranking of returned scalar values. The platform calls this function once per candidate input, sorts by score, reveals the top K inputs per data category (ties broken uniformly at random), and passes those labeled inputs into the participant's predict() as its labeled argument (a list of dicts carrying the same four fields plus a "label" key in {0,1}). If no acquisition function is provided, K inputs per data category are chosen uniformly at random instead, so every submission has a well-defined default. K is set per data category and controlled by the organizers (default $K = 5$).

Total label budget per round. With m data categories present in the sampled hidden slice (Section 2.1) and K revealed labels per data category, each submission receives at most $K \cdot m$ labeled inputs before any call to predict(). In the active configuration, $K = 5$ and m is the number of available data categories in that submission's sample. The labeled list may be shorter if a data category has fewer eligible pairs, and it may be empty in local/offline smoke tests where the data-service adaptive-label endpoint is not available. These labels are then available to every subsequent predict() call in that run. Across rounds the labeled set is resampled (with a different

candidate pool and different top- K selections), so submissions cannot accumulate labels across the competition.

Fallback on invalid acquisition output. If `acquisition_function()` raises an exception, exceeds its per-call timeout, or returns a non-finite value (`NaN`, $\pm\infty$) for *any* candidate in a round, the platform discards all scores from that round and falls back to uniformly random selection of K labeled inputs per data category, the same default that applies to submissions without a `labeling.py`. Ties within a valid ranking are broken uniformly at random. The submission’s `predict()` is still invoked; the only consequence of a fallback is that its `labeled` argument carries a random sample rather than a learned-acquisition sample. Students should test their acquisition function locally against all four-field inputs produced from the public training set before submitting.

Exact signatures and a reference implementation are given in the Starter Kit (Section 3.3).

3 Submission and Grading

3.1 Grading

Collaboration in teams of 1–3 students is permitted. Each team submits once per grading window. Inter-team discussion on ideas is encouraged (e.g., discussing approaches on Ed), but each team must submit their own code and predictions independently. Teams form freely during the first two weeks of the competition; at the end of Week 2 the team roster is locked. After the lock date, merges between existing teams require instructor approval and are granted only in exceptional cases (e.g., a teammate withdraws from the course or becomes unresponsive). No new teams may form in the final two weeks before the deadline. Grading combines leaderboard performance with a written report to reward both strong results and deep understanding:

- Technical report (50%). Each team submits a short report on Gradescope describing their approach, including: (1) problem formulation and method description, (2) description of the training dataset that they choose to use, (3) experimental results and ablation studies on the validation set, and (4) analysis of failure modes or interesting patterns in the data. Reports must use the NeurIPS 2025 L^AT_EX template (<https://neurips.cc/Conferences/2025/CallForPapers>) and are limited to 4 pages of main content; references and appendices do not count toward the page limit but graders are not obligated to read the appendix. Reports are graded on clarity, depth of analysis, and insight.
- Leaderboard performance (50%). Each team’s score is the negative log-loss (higher is better) of its best-scoring submission on the private test set, taken as the maximum over all rounds submitted during the competition window. Grading is anchored to an organizer-provided baseline: beating the baseline guarantees at least an 80 percent score on this component, and additional points are awarded based on leaderboard ranking among teams that also beat the baseline. Submissions that do not beat the baseline receive partial credit based on how close they come to it (and on the report quality). The baseline’s private test score will be released three weeks after the start of the competition.

In addition to the report, each team uploads its *final code* to Gradescope, specifically the `model.py` and (if used) `labeling.py` corresponding to the best-scoring Codabench submission, along with any auxiliary files needed to reproduce that run (e.g., `models.txt`, weight checkpoints under the size cap, a `README.md` with instructions). Gradescope is the authoritative record for grading: if the code

on Gradescope does not correspond to the submission that produced the team’s best Codabench score, the Gradescope version is used. We use Gradescope because it is Stanford-approved for course submissions and gives us a stable, archival record of each team’s final work.

3.2 Submission format

The participant uploads a ZIP archive to Codabench (<https://aimslab.stanford.edu/competition/submit>) containing `model.py` (required), and optionally `labeling.py`, `requirements.txt`, and `models.txt`. The ZIP format accepts `requirements.txt`, but runtime installation of additive packages is organizer-controlled and disabled by default in the example deployment. The `models.txt` file declares HuggingFace model repositories that the submission needs at runtime. The platform executes submitted code in a sandboxed container; the starter-kit helpers described below hide the container’s file layout so that student code only needs to reason about items, subjects, and probabilities.

3.3 Starter kit

We provide a starter kit on Codabench (`starting_kit/`) that includes an end-to-end example submission, a CPU smoke-test script, and reference baselines (majority-class, per-subject mean accuracy, and a content-based embedding model). Students should start from `sample_code_submission/` and only need to implement one required entry point in `model.py` and one optional entry point in `labeling.py`.

An *input* is a dict describing one item-subject-benchmark-condition combination, with exactly these four string-valued keys:

Key	Value (all <code>str</code>)
"benchmark"	Benchmark name (e.g., "MMLU", "GSM8K").
"condition"	Test condition (e.g., "zero-shot"); the literal string "none" when no condition applies.
"subject_content"	Description of the AI subject being evaluated (name, organization, size, release date, etc.).
"item_content"	The question, prompt, or task text that the subject is asked to respond to.

The platform may internally split one upstream benchmark item into multiple hidden item variants when that item appears under multiple test conditions. Participants do not need to reconstruct this mapping: the runtime `item_id` in `test_pairs.csv` already identifies the specific variant, while `input["condition"]` tells the submission which condition applies. If no condition applies, the value is the literal string "none". The prediction CSV contract remains `model_id,item_id,predicted_probability`; submissions are scored against `model_id,item_id,label` ground truth by the same runtime identifiers.

A *labeled input* is the same dict with one extra key:

Key	Value
"label"	<code>int</code> in $\{0,1\}$, encoded as 1 if the subject passed the item and 0 otherwise; the ground-truth label revealed by the adaptive-labeling mechanism.

```

# model.py (required)
def predict(input: dict,
            labeled: list[dict] | None = None) -> float:
    """
    input: dict with keys "benchmark", "condition",
           "subject_content", "item_content" (all Python str).
    labeled: list of dicts with the same four keys plus "label"
             (Python int in {0, 1}) revealed by the adaptive-
             labeling mechanism. May be empty or None. The same
             'labeled' list is passed on every call within a round.
    Returns: a native Python 'float' in [0, 1], the probability
             that the subject passes this item. Wrap any numpy or
             torch scalar in float(...) before returning, e.g.,
             'return float(tensor.item())'; numpy/torch scalars
             are not accepted and will fail CSV serialization.
    """

# labeling.py (optional, for adaptive labeling)
def acquisition_function(input: dict) -> float:
    """
    input: dict with the same four string keys as above, whose
           ground-truth label could be revealed this round.
    Returns: a native Python 'float', finite and non-NaN. Higher
             = more desired for labeling. Wrap any numpy or torch
             scalar in float(...) before returning. The platform
             calls this once per candidate input, reveals labels
             for the top-K-scoring inputs per data category, and
             passes them as 'labeled' to predict(). Ties are broken
             uniformly at random; NaN/inf trigger the random-
             selection fallback (see Section~\ref{sec:adaptivity}).
    """

```

Training data lives on the public HuggingFace dataset (see Section 2.1) with the same four string fields; students download it and preprocess it however they prefer before submitting. Small fitted state (fine-tuned adapter weights, embedding caches, lookup tables) should be baked into the submission ZIP. Large checkpoints, such as an LLM the submission uses as an encoder or judge, should instead be uploaded to a HuggingFace model repository and declared in `models.txt`, which the platform pre-fetches into the container. In both cases the model should be loaded at the top level of `model.py` (i.e., outside any function body), not inside `predict()`. When the platform starts a submission container, it imports `model.py` once, and `predict()` is then called many times, once per input (i.e., a dictionary). Loading at the top level means the load cost, which for a large checkpoint can be several minutes, is paid only once when the container starts, and every subsequent call to `predict()` reuses the already-loaded weights. Loading inside `predict()` would instead reload on every call and usually exceeds the per-round time budget.

The submission sandbox is *network-isolated at test time*. Neither `predict()` nor `acquisition_function()` may make outbound network calls to any third-party endpoint; this includes hosted LLM APIs, remote embedding services, external object storage (S3, GCS, Azure Blob), remote databases, webhooks, your own cloud functions, or anything else that would let the submission communicate with the outside world during an evaluation round. Any model the submission wants to use must be either baked into the ZIP or declared in `models.txt` and pre-fetched by the platform before the container starts. Each submission also runs in a fresh

container that is destroyed when the round ends, so module-level state does not persist across rounds.

The rule serves three purposes. First, it keeps the competition about *predicting* AI performance rather than *measuring* it on the fly by routing test items to an external model. Second, and equally important, it closes an external-storage side-channel: without this rule a submission could stash seen test inputs or implied labels on a remote server during one round and pull them back on the next, accumulating information the round-to-round resampling is explicitly designed to prevent. Third, it ensures all teams compete on comparable compute; the only resources available to a submission are those baked into its ZIP or declared in `models.txt`.

Each round, the platform calls `acquisition_function()` if `labeling.py` is present, reveals labels for the top- K scored inputs per data category, and then calls `predict()` with those labeled inputs. In the active configuration, $K = 5$ and the sampled hidden slice uses all available data categories, so the `labeled` list contains at most 5 labeled dicts per data category present in that run. If no `labeling.py` is shipped, K inputs per data category are chosen uniformly at random instead, so every submission has a well-defined default. If `acquisition_function()` raises an exception, times out, or returns a non-finite value (NaN or $\pm\infty$) for any candidate, the platform falls back to the random-selection default for that round; `predict()` is still called, but its `labeled` argument carries the random sample rather than the ranked one. See Section 2.2 for the exact rule.

3.4 Worked example: content-based neural collaborative filtering

To make the signatures concrete, the starter kit includes a hypothetical implementation of a content-based neural collaborative filtering (NCF) baseline. The idea: embed the subject and item fields with an off-the-shelf *sentence transformer*, a pretrained neural network that maps an arbitrary text string to a fixed-length dense vector whose geometry reflects semantic content (e.g., `all-mpnet-base-v2` produces 768-dimensional vectors), then feed the two vectors through a small MLP head whose weights are trained offline on the public training data. Because both subject and item are represented by their text content alone, the method works in the cold-start regime by design.

```
# model.py (schematic pseudocode)
from sentence_transformers import SentenceTransformer
import torch

# Runs once when the container starts
ENCODER = SentenceTransformer("all-mpnet-base-v2")
NCF = torch.load("ncf_head.pt") # MLP trained offline
NCF.eval()

# Runs once per test input
def predict(input, labeled=None):
    u = ENCODER.encode(input["subject_content"], convert_to_tensor=True)
    v = ENCODER.encode(input["item_content"], convert_to_tensor=True)
    with torch.no_grad():
        logit = NCF(torch.cat([u, v]))
    return torch.sigmoid(logit).item()
```

Offline, on their own machine, the student loads the HuggingFace training set (which carries the same four fields), precomputes subject and item embeddings (with caching so each unique string is encoded once), and fits the MLP:

```
# train_ncf.py (run once, offline; writes ncf_head.pt)
```

```

from datasets import load_dataset
import torch.nn as nn

df = load_dataset("aims-foundations/measurement-db", split="train")
U = ENCODER.encode(df["subject_content"], batch_size=256)
V = ENCODER.encode(df["item_content"], batch_size=256)

ncf = nn.Sequential(
    nn.Linear(2 * d, 256), nn.ReLU(),
    nn.Linear(256, 256), nn.ReLU(),
    nn.Linear(256, 1),
)

for (u, v, y) in batches(U, V, df["label"]):
    loss = bce_with_logits(ncf(torch.cat([u, v], dim=-1)), y)
    step(loss)

torch.save(ncf, "ncf_head.pt")  # bundle into the submission ZIP

```

The labeled argument of `predict` is unused in this baseline, so it inherits whatever the default random selection provides. A stronger variant uses it for online calibration: at the first call of each round, fit a one-parameter Platt scaling on the revealed labeled dicts (each carrying the four fields plus a "label") and cache the calibrator on a module-level variable, then pass every subsequent NCF score through it before returning. This is a good way to demonstrate the value of the adaptive-labeling channel in the technical report.

3.5 Worked example: local LLM-as-judge

Because the four input fields are all text, they can be assembled into a prompt for a language model without any external preprocessing. The starter kit therefore also includes an LLM-as-judge baseline: ask a *local* open-weights model whether the subject would pass the item, and read the probability off of the token-level log-probabilities rather than a free-form numeric reply. Since outbound network calls are prohibited from `predict()`, the judge model must be declared in `models.txt` (so the platform pre-fetches it into the HF cache before the container starts) and loaded from disk at module scope.

```

# model.py    (schematic pseudocode)
import math, torch
from transformers import AutoTokenizer, AutoModelForCausalLM

# Runs once when the container starts
# The HF repo below is declared in models.txt and pre-fetched by
# the platform, so this load hits the local HF cache only.
MODEL_ID = "Qwen/Qwen2.5-7B-Instruct"
TOK = AutoTokenizer.from_pretrained(MODEL_ID)
MODEL = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, torch_dtype=torch.bfloat16, device_map="auto"
)
MODEL.eval()

YES_ID = TOK.encode(" yes", add_special_tokens=False)[-1]
NO_ID = TOK.encode(" no", add_special_tokens=False)[-1]

```



```

JUDGE = """You will see a description of an AI subject and an
evaluation item. Decide whether the subject would answer the item
correctly. Reply with a single token: yes or no.

Benchmark: {benchmark}
Condition: {condition}
Subject:   {subject_content}
Item:      {item_content}

Answer: """

# Runs once per test input
def predict(input, labeled=None):
    ids = TOK(JUDGE.format(**input),
              return_tensors="pt").to(MODEL.device)
    with torch.no_grad():
        logits = MODEL(**ids).logits[0, -1]    # next-token logits
        lp     = torch.log_softmax(logits, dim=-1)
        p_yes  = math.exp(lp[YES_ID].item())
        p_no   = math.exp(lp[NO_ID].item())
    return p_yes / (p_yes + p_no)              # renormalize over {yes, no}

```

Two design choices are worth pointing out in the report:

- *Why logprobs and not a numeric reply?* Asking the LLM for “a probability between 0 and 1” is brittle: models round, miscalibrate, and occasionally emit non-numeric text. Reading the next-token log-probabilities of the **yes/no** tokens gives a calibrated binary probability for free, which is what the competition’s log-likelihood metric rewards.
- *Using `labeled` for in-context calibration.* Revealed labeled dicts can be rendered through the same JUDGE template (with their “label” spelled out as **Answer: yes/no**) and prepended to the judge prompt as few-shot examples. A typical pattern is to bucket the labeled inputs by their “benchmark” field and include two to four same-benchmark examples ahead of the test input. The technical report should ablate this (zero-shot vs. few-shot) to quantify the value of adaptive labeling for the LLM-as-judge pipeline.

Operationally, `predict()` is invoked once per hidden runtime model-item pair. The active hidden sample contains 5,000 item variants, and the number of prediction calls equals the number of runtime pairs across those variants and subjects. A local 7B-class judge therefore needs careful caching, batching, quantization (`bitsandbytes`, `AWQ`), or a smaller backbone to fit the per-round budget. Students who want to batch multiple inputs into one forward pass can buffer them on a module-level queue and flush when the buffer is full, returning cached probabilities on subsequent calls within the round.

3.6 Worked example: k -means diversity acquisition function

The simplest acquisition strategy to reason about is *diversity sampling*: cluster the input space offline, then at test time reveal labels for inputs that cover clusters which the round has not yet sampled from. Intuitively, if we are going to buy only K labels per data category, they should be spread across the kinds of prompts the round contains, not concentrated in one region of embedding space.

A subtlety worth calling out up front: `acquisition_function()` is called once per candidate input with no access to the full candidate list, so any strategy that needs cross-candidate information (e.g., diversity, k -center) must accumulate state on module-level variables as the platform iterates through the candidates. The candidate pool for each round is bounded and the platform calls the function in a single pass, so a simple dictionary keyed by a stringified version of the input (e.g., `json.dumps(input, sort_keys=True)`) is sufficient.

The student picks a concatenation template, any fixed function of the four fields will do as long as the same function is used at train and test time, and fits k -means on sentence embeddings of the public training set:

```
# fit_centroids.py (run once, offline; writes centroids.npy)
import numpy as np
from sklearn.cluster import KMeans
from datasets import load_dataset
from sentence_transformers import SentenceTransformer

ENCODER = SentenceTransformer("all-mpnet-base-v2")

def to_text(ex: dict) -> str:
    """Same function is used offline and at test time."""
    return (f"Benchmark: {ex['benchmark']}\n"
            f"Condition: {ex['condition']}\n"
            f"Subject: {ex['subject_content']}\n"
            f"Item: {ex['item_content']}\n")

df = load_dataset("aims-foundations/measurement-db", split="train")
X = ENCODER.encode([to_text(r) for r in df], batch_size=256)
km = KMeans(n_clusters=64, n_init=10).fit(X)

np.save("centroids.npy", km.cluster_centers_)
```

At test time, each candidate is assigned to its nearest centroid; scoring prefers clusters that have been visited less often so far in the round:

```
# labeling.py (schematic pseudocode)
from collections import Counter
import numpy as np
from sentence_transformers import SentenceTransformer

# Runs once when the container starts
ENCODER = SentenceTransformer("all-mpnet-base-v2")
CENTROIDS = np.load("centroids.npy") # shape: (K, d)

def to_text(ex: dict) -> str:
    return (f"Benchmark: {ex['benchmark']}\n"
            f"Condition: {ex['condition']}\n"
            f"Subject: {ex['subject_content']}\n"
            f"Item: {ex['item_content']}\n")

_seen = Counter() # per-cluster candidate count for this round

# Runs once per candidate
def acquisition_function(input):
    x = ENCODER.encode(to_text(input))
```

```

c = int(np.argmin(np.linalg.norm(CENTROIDS - x, axis=1)))
_seen[c] += 1
return -_seen[c]    # first-in-cluster scores highest

```

With this scoring rule, every candidate that is first-of-its-cluster in the round’s iteration order receives the same top score (-1); ties are broken uniformly at random (see Section 2.2), so the platform ends up sampling one representative per cluster when K is not larger than the number of occupied clusters. When K exceeds the cluster count, the rule smoothly falls back to drawing the second representative from each cluster, then the third, and so on.

One simple variant is *online farthest-point sampling*: skip the offline k -means entirely and instead keep a module-level list of already-encoded candidates, returning the distance from the current input to its nearest previously-seen neighbor. The first candidate seeds the pool; later candidates score high only if they are dissimilar to everything seen so far. This achieves diversity without shipping any pre-fit centroids.

4 Other Logistics

1. Participants may use any method, including but not limited to IRT models, collaborative filtering, matrix factorization, deep learning, and ensemble approaches.
2. Participants may use any publicly available metadata (such as model names, parameter counts, release dates, item text, benchmark categories). Participants are encouraged to curate more data for the training process.
3. Modal provides a \$500 GPU computing credit to each enrolled student through an individual coupon code. The credit is granted directly by Modal and cannot be pooled, reissued, or reassigned by the course staff. Team members may naturally apply their individual credits to shared work, so a three-person team has effectively \$1,500 of Modal budget versus \$500 for a solo competitor. This is a real disparity in raw compute, but in practice a well-designed baseline uses only a small fraction of \$500: the leaderboard rewards efficient, content-based predictors, not expensive inference, and larger teams also pay coordination costs and share a single team grade. We recommend choosing your team size on pedagogical grounds (who you want to work with, how you want to divide the work) rather than on expected compute consumption.
4. The organizers will provide computing resources to host the competition leaderboard and to automate the evaluation of submissions. Depending on the computational complexity of the submission, the feedback may not be instantaneous.
5. We provide the `torch_measure` package to facilitate implementation of the measurement model using GPU-enabled PyTorch. Using the package is not required for competition participation.
6. The submission sandbox is network-isolated at test time: no outbound calls from `predict()` or `acquisition_function()` to any third-party endpoint (hosted LLM APIs, remote embedding services, external object storage, remote databases, webhooks, or anything else). Submissions also run in a fresh container that is destroyed when the round ends, so no state persists across rounds. All models used at test time must be bundled into the submission ZIP or declared in `models.txt` for the platform to pre-fetch. Hosted APIs may still be used offline during training data curation and model fitting.

7. Submitted code undergoes manual review to verify integrity and rule compliance.